

Exercises

AGGREGATE MODELLING

E/R schema and Workload

Q1: Given a product, display its category and all orders that contain it

Q2: Update a product

Normalized Solution : Category, Product, Order, Item

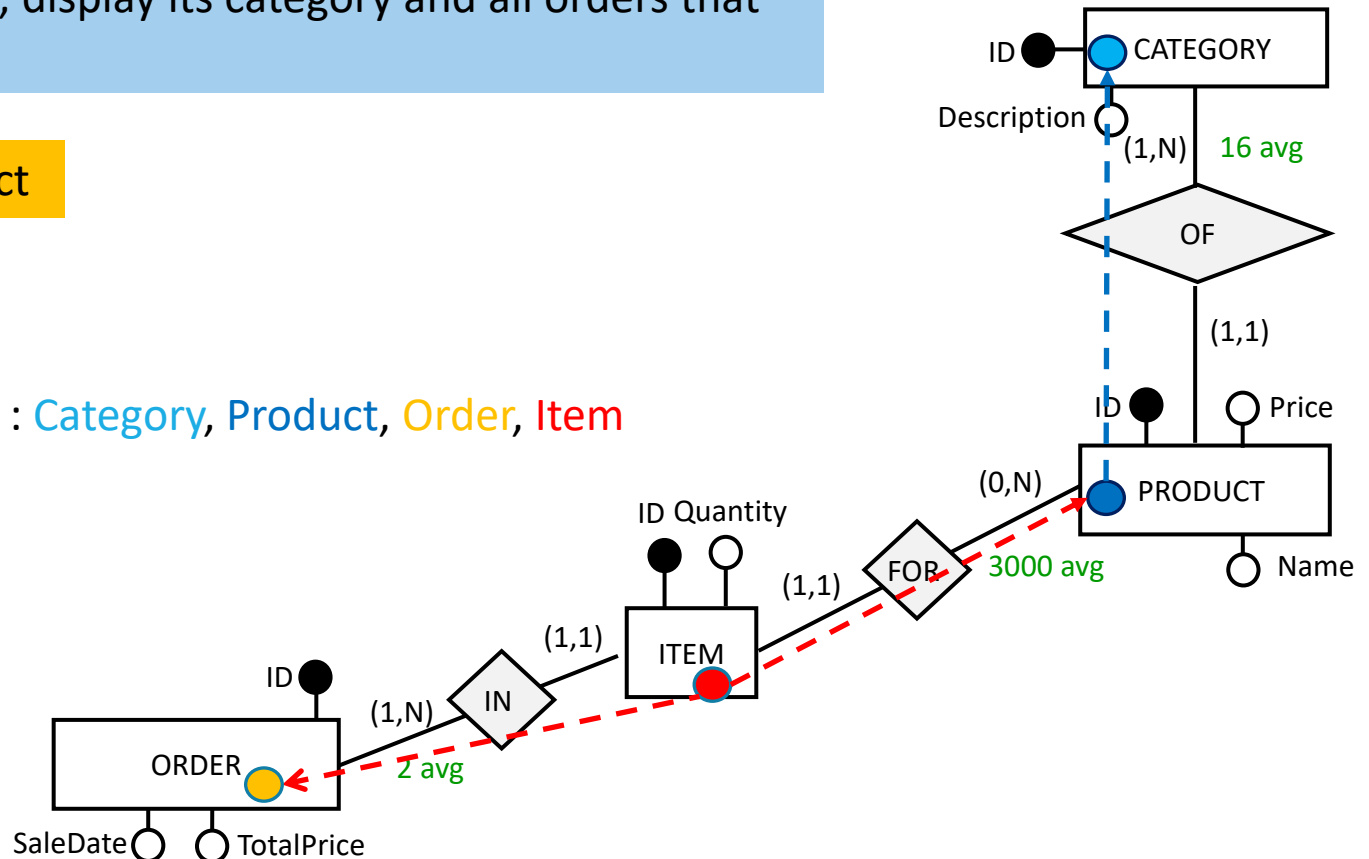
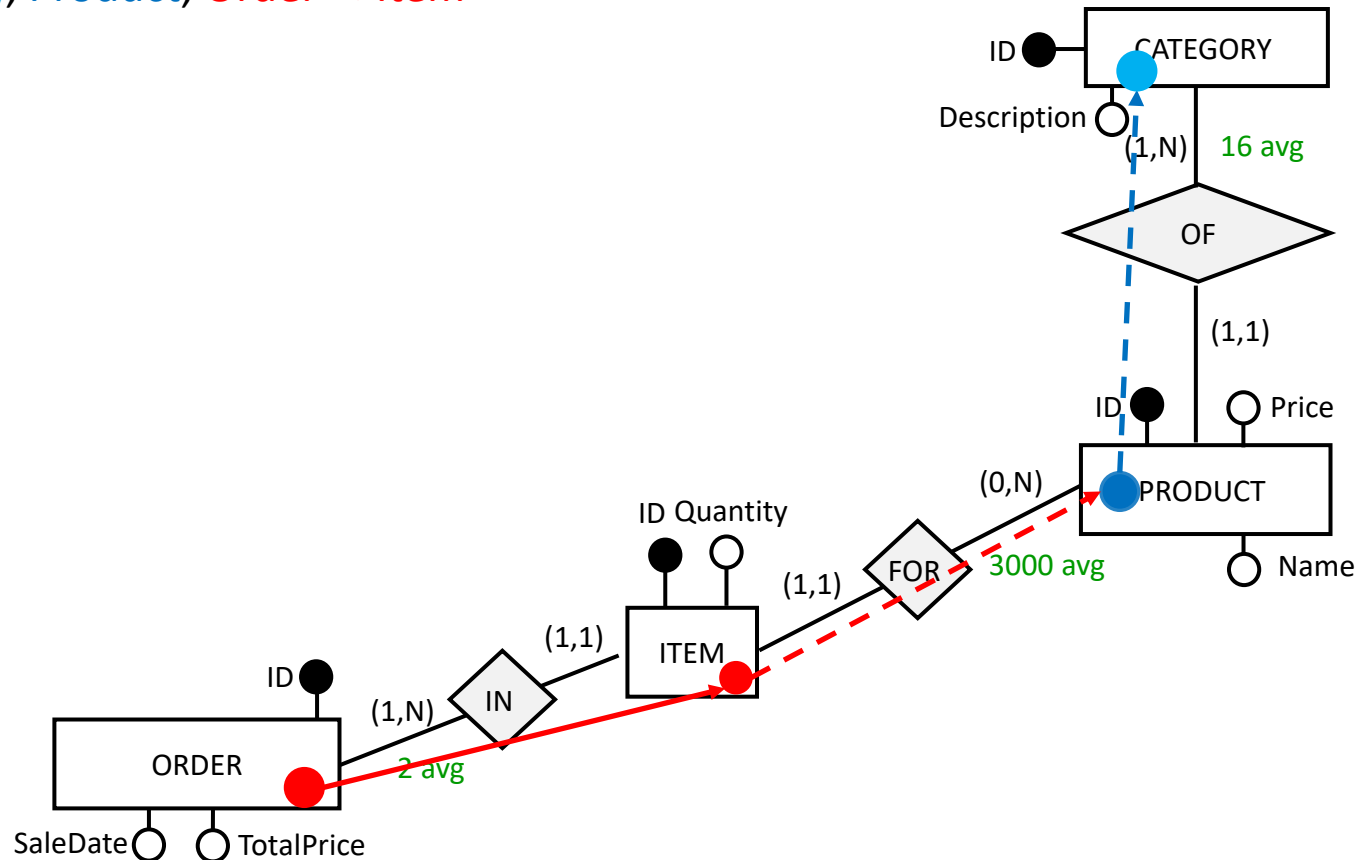


Table of Volumes

Name	ND	len
item	2.400.000	100
product	800	135
order	1.200.000	350
category	50	100

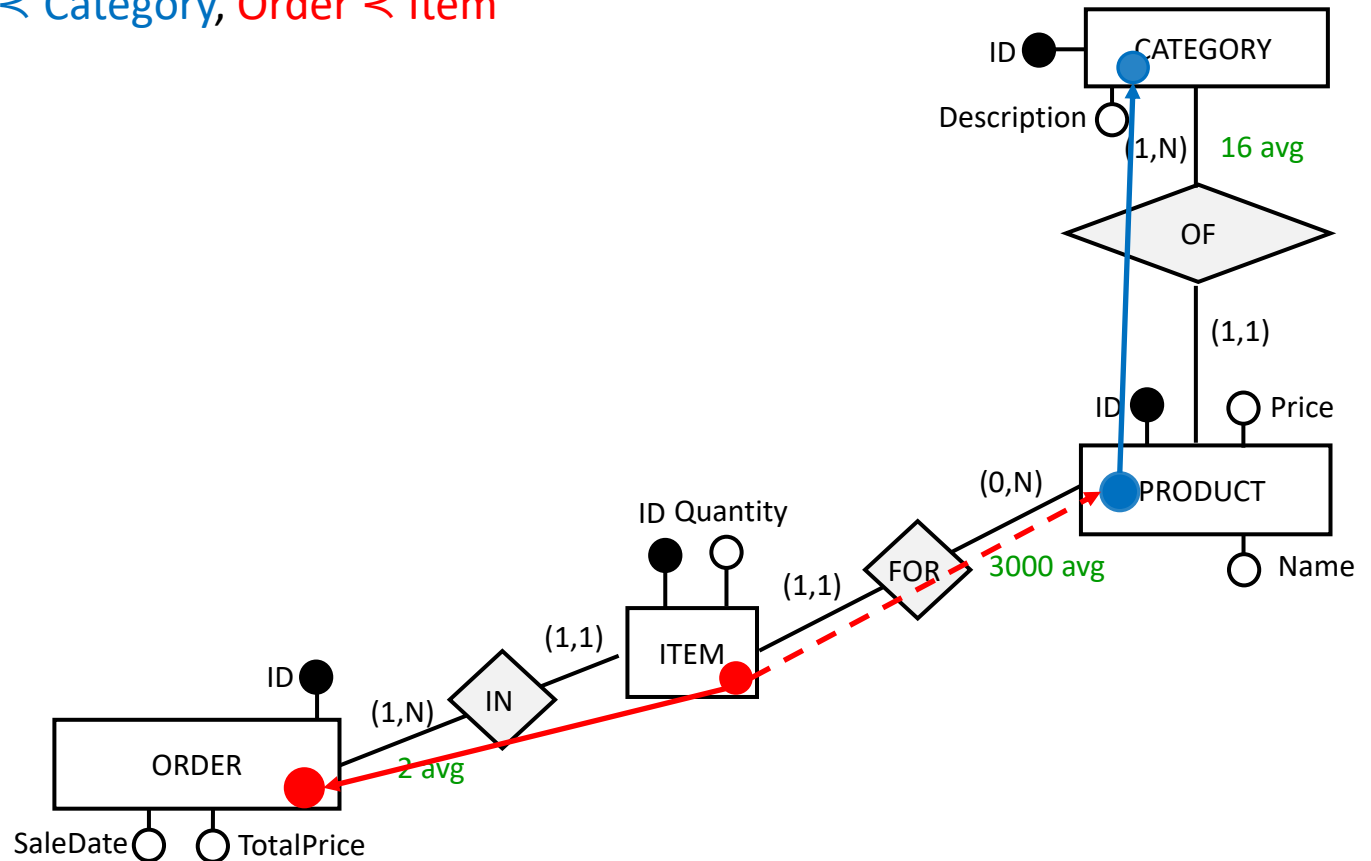
Solution 1

Collections: **Category**, **Product**, **Order** < **Item**



Solution 2

Collections: **Product** < **Category**, **Order** < **Item**



Exercise 1

1. Load the SmallOnlinestore DB from DUMP (from virtuale)
 - `mongorestore --db SmallOnlinestore <dumppath>`
2. Write the queries to create the collections for solution 1 and 2.

```
db.orders.aggregate([
  { $lookup: { from: 'item', localField: 'id',
    foreignField: 'orders', as: 'items' } },
  { $out: { db: "SmallOnlinestore", coll: "s1.orders" } },
]);
```

- **s2.product:** ...

3. Estimate the size of the solutions
4. Write the queries 1-2 for both the solutions
5. Estimate the cost of the workload in both the solutions

Step 2: DB creation

```
// s1.orders
```

```
db.orders.aggregate([
{$lookup: { from: 'item', localField: 'id', foreignField:
'orders', as: 'items' } },
{ $out: { db: "SmallOnlinestore", coll: "s1.orders" } }, 1]);
```

```
// s2.product
```

```
db.product.aggregate([
{$lookup: { from: 'category', localField: 'category',
foreignField: 'id', as: 'cat' } },
{$project: { _id: 1, id: 1, price: 1, supplier: 1, description:1,
category: { $arrayElemAt: ["$cat", 0] }, }},
{ $out: { db: "SmallOnlinestore", coll: "s2.product" } }, 1]);
```

Step 3: Table of Volumes

SOLUTION 1	len	ND	NP
Category	100	50	2
Product	145	800	42
Order	570	1200000	242018
TOTAL			242062

SOLUTION 2	len	ND	NP
Product	235	800	67
Order	570	1200000	242018
TOTAL			242085

```
var stats = db.getSiblingDB("SmallOnlinestore").order.stats();
var selectedFields = {
  avgObjSize: stats.avgObjSize,
  count: stats.count,
  storageSize: stats.storageSize,
  totalIndexSize: stats.totalIndexSize,
  totalSize: stats.totalSize,
  nindexes: stats.nindexes,
};

printjson(selectedFields);
```

Step 4: Queries

```
//Q1: Given a product, display its category and all orders that contain it
db.product.aggregate([
  { $match: { id: 1 } },
  { $lookup: { from: "category", localField: "id", foreignField:
"items.product", as: "orders" } },
  { $lookup: { from: "s1.orders", localField: "id", foreignField:
"items.product", as: "orders" } },
]);

db.s2.product.aggregate([
  { $match: { id: 1 } },
  { $lookup: { from: "s1.orders", localField: "id", foreignField:
"items.product", as: "orders" } },
  { $unwind: "$orders" },
  { $unwind: "$orders.items" },
]);

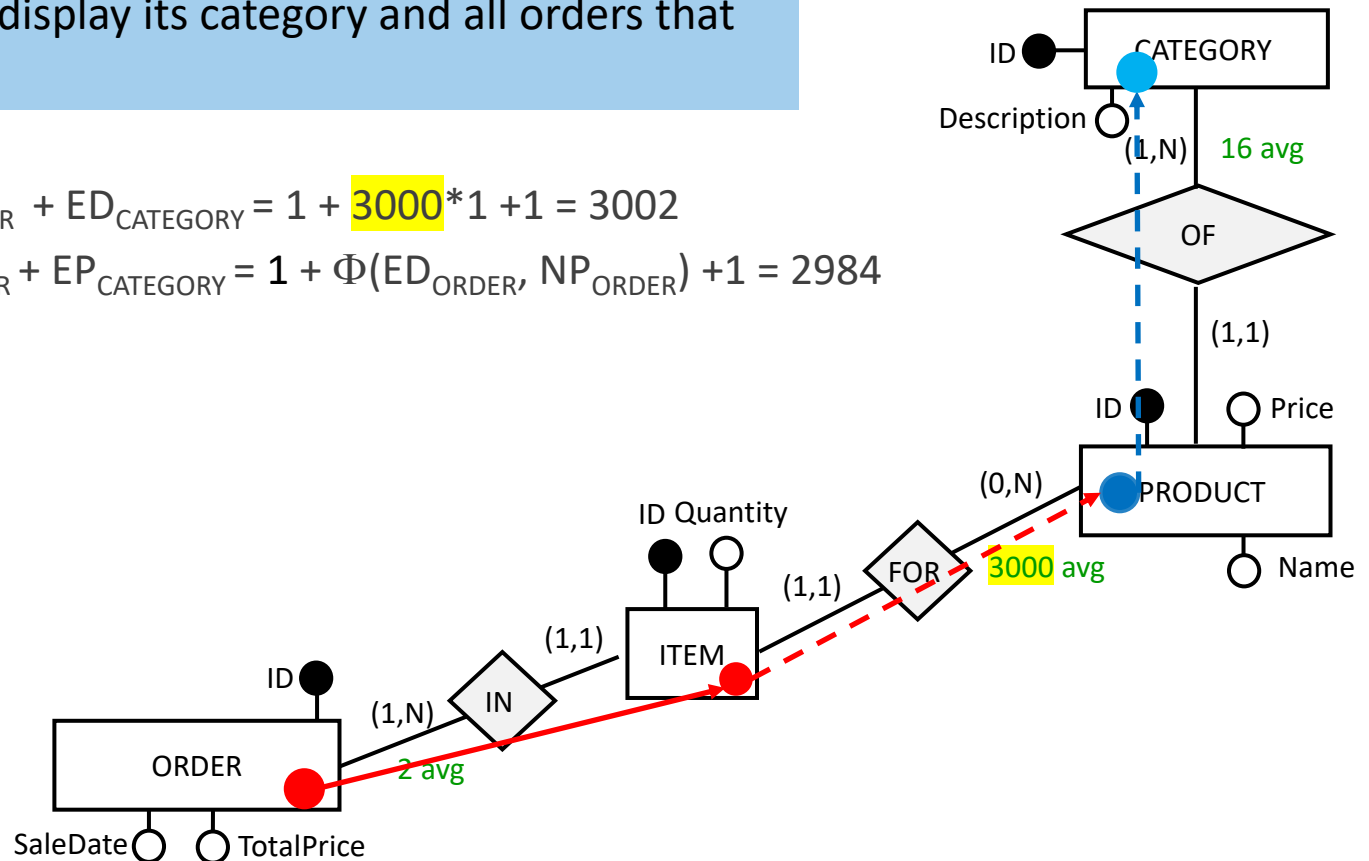
// Q2: Update a product
db.product.updateOne({ id: 1 }, { $set: { price: 1000 } });
```

Step 5: Q1 cost Solution 1

Q1: Given a product, display its category and all orders that contain it

$$ED = ED_{\text{PRODUCT}} + ED_{\text{ORDER}} + ED_{\text{CATEGORY}} = 1 + 3000 * 1 + 1 = 3002$$

$$EP = EP_{\text{PRODUCT}} + EP_{\text{ORDER}} + EP_{\text{CATEGORY}} = 1 + \Phi(ED_{\text{ORDER}}, NP_{\text{ORDER}}) + 1 = 2984$$

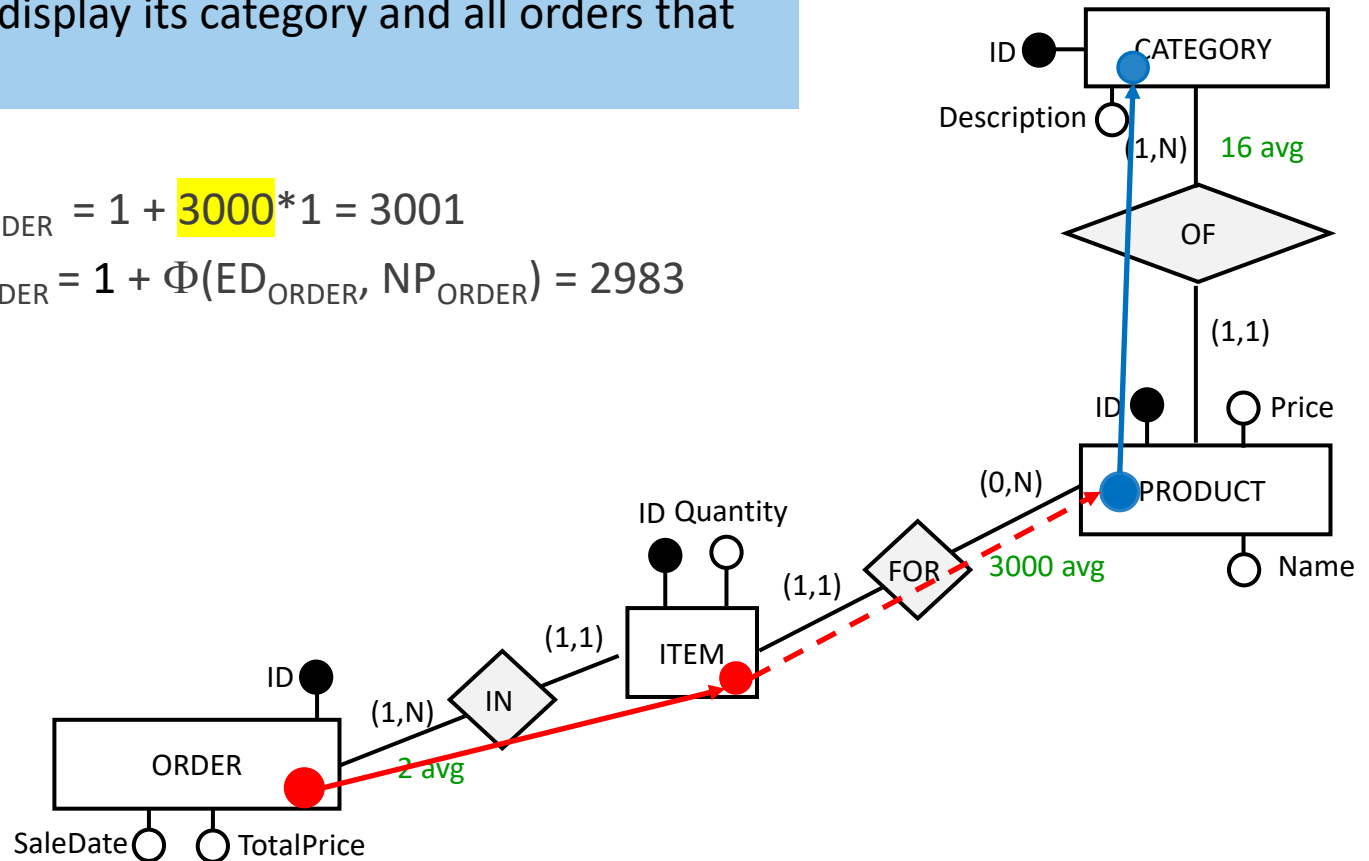


Step 5: Q1 cost Solution 2

Q1: Given a product, display its category and all orders that contain it

$$ED = ED_{\text{PRODUCT}} + ED_{\text{ORDER}} = 1 + 3000 * 1 = 3001$$

$$EP = EP_{\text{PRODUCT}} + EP_{\text{ORDER}} = 1 + \Phi(ED_{\text{ORDER}}, NP_{\text{ORDER}}) = 2983$$



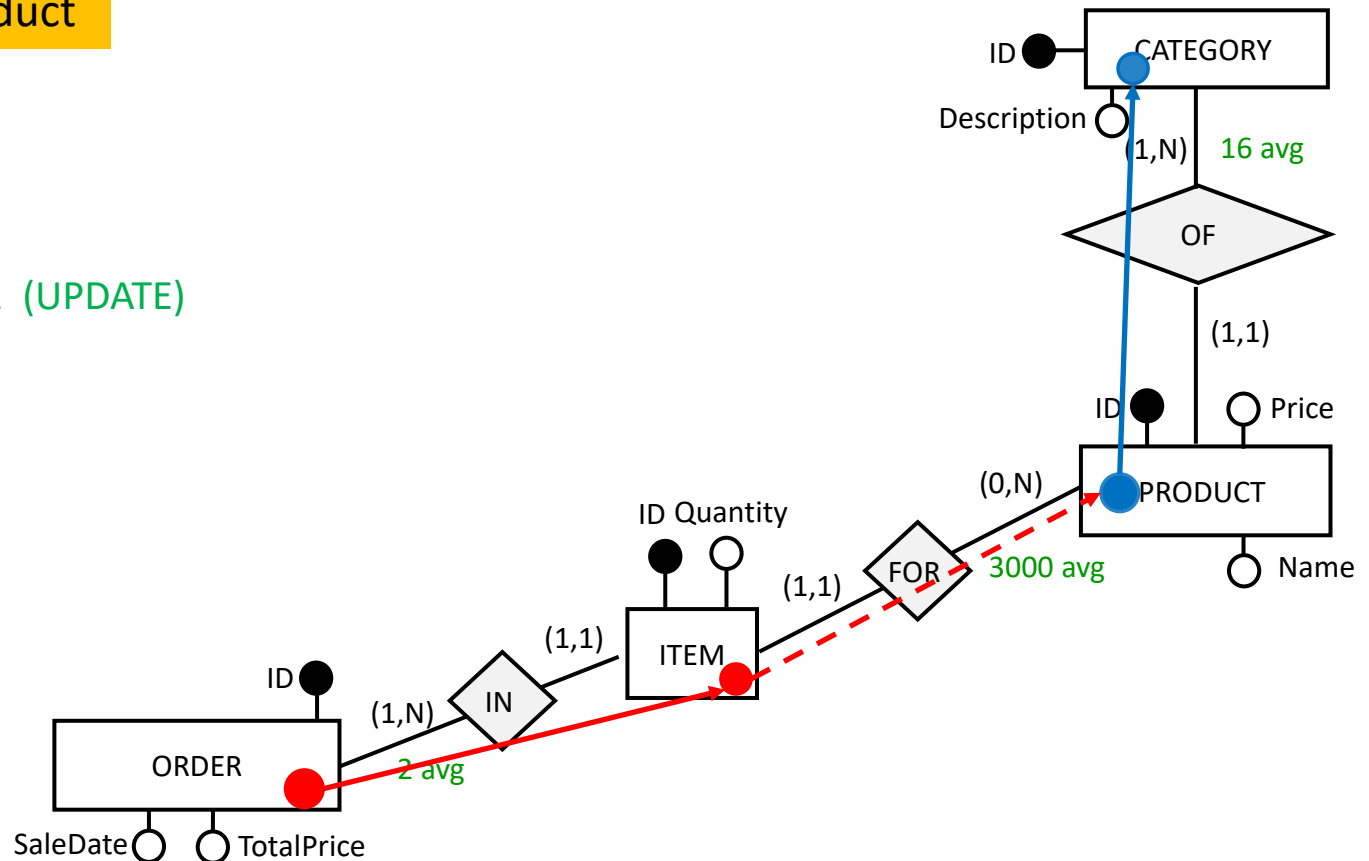
Step 5: Q2 cost Solutions 1&2

Q2: Update a product

$$ED = ED_{\text{PRODUCT}} = 1$$

$$EP = EP_{\text{PRODUCT}} = 1$$

$$\text{TotalCost} = 2 * EP = 2 \text{ (UPDATE)}$$



Summary

	Max len	NP	Tot ED	Tot EP
Solution 1	570	242062	3003	2986
Solution 2	570	242085	3002	2985

E/R schema and Workload

Q3: Given a order, display all items that it contains (including price and category)

Q4: Update an order

Normalized Solution : Category, Product, Order, Item

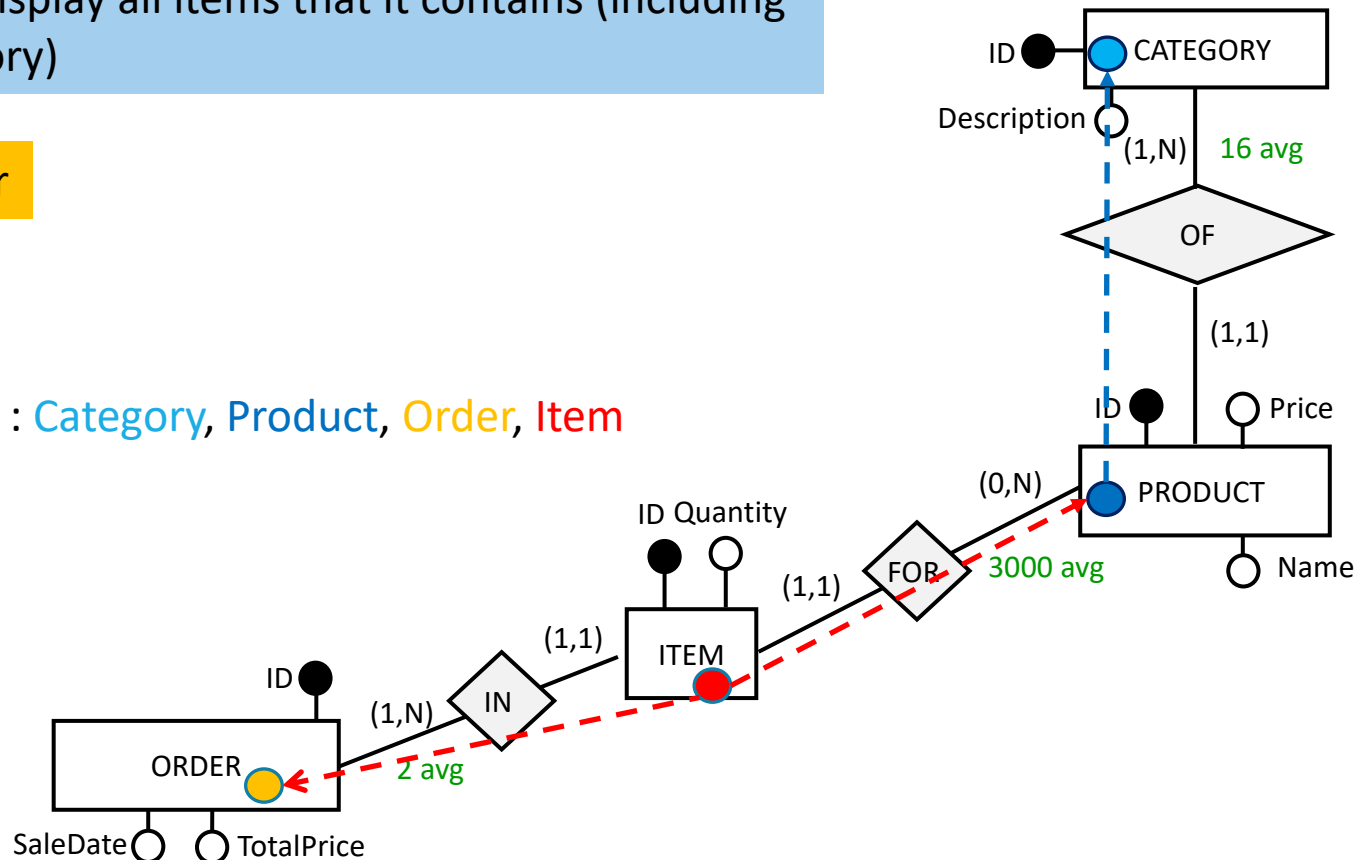
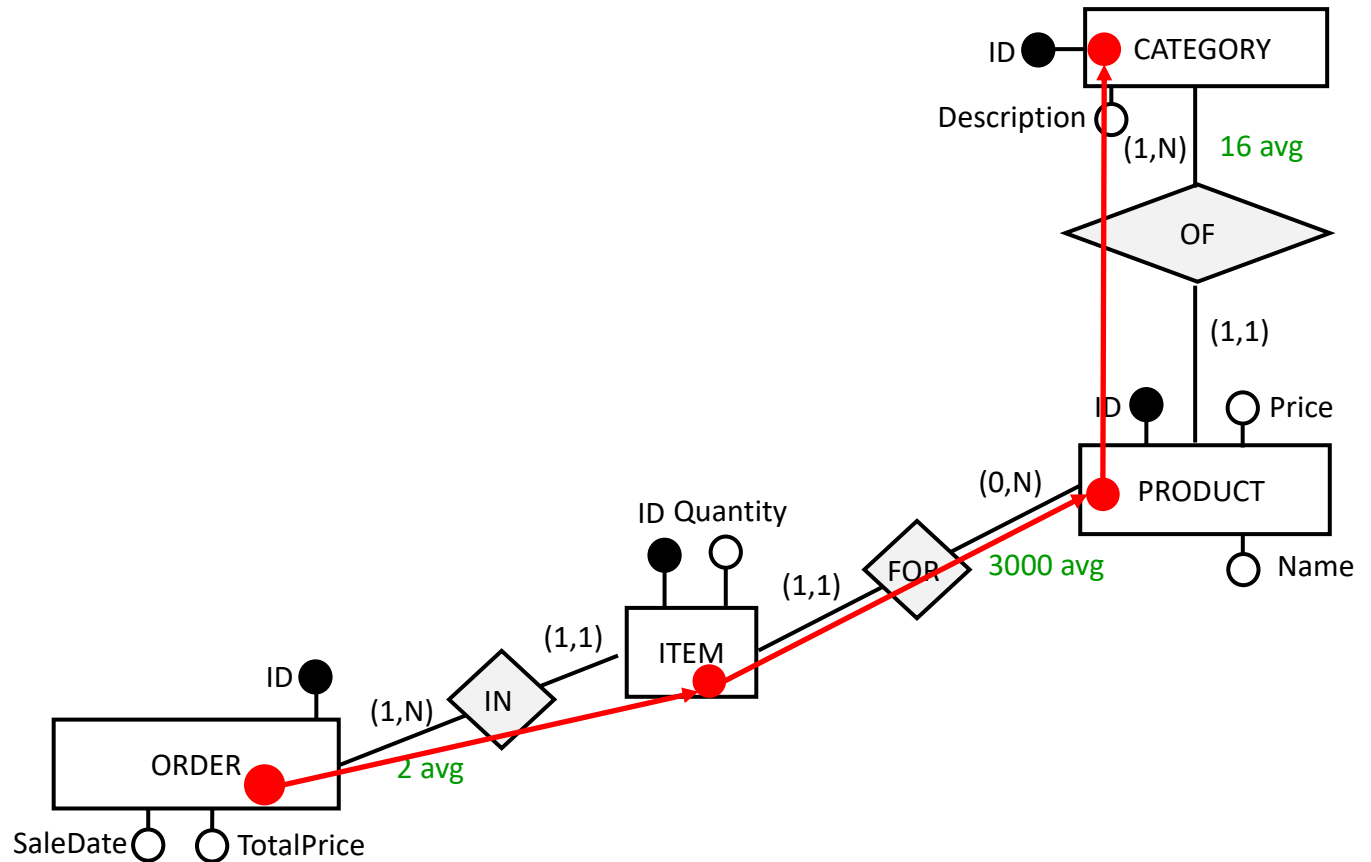


Table of Volumes

Name	ND	len
item	2.400.000	100
product	800	135
order	1.200.000	350
category	50	100

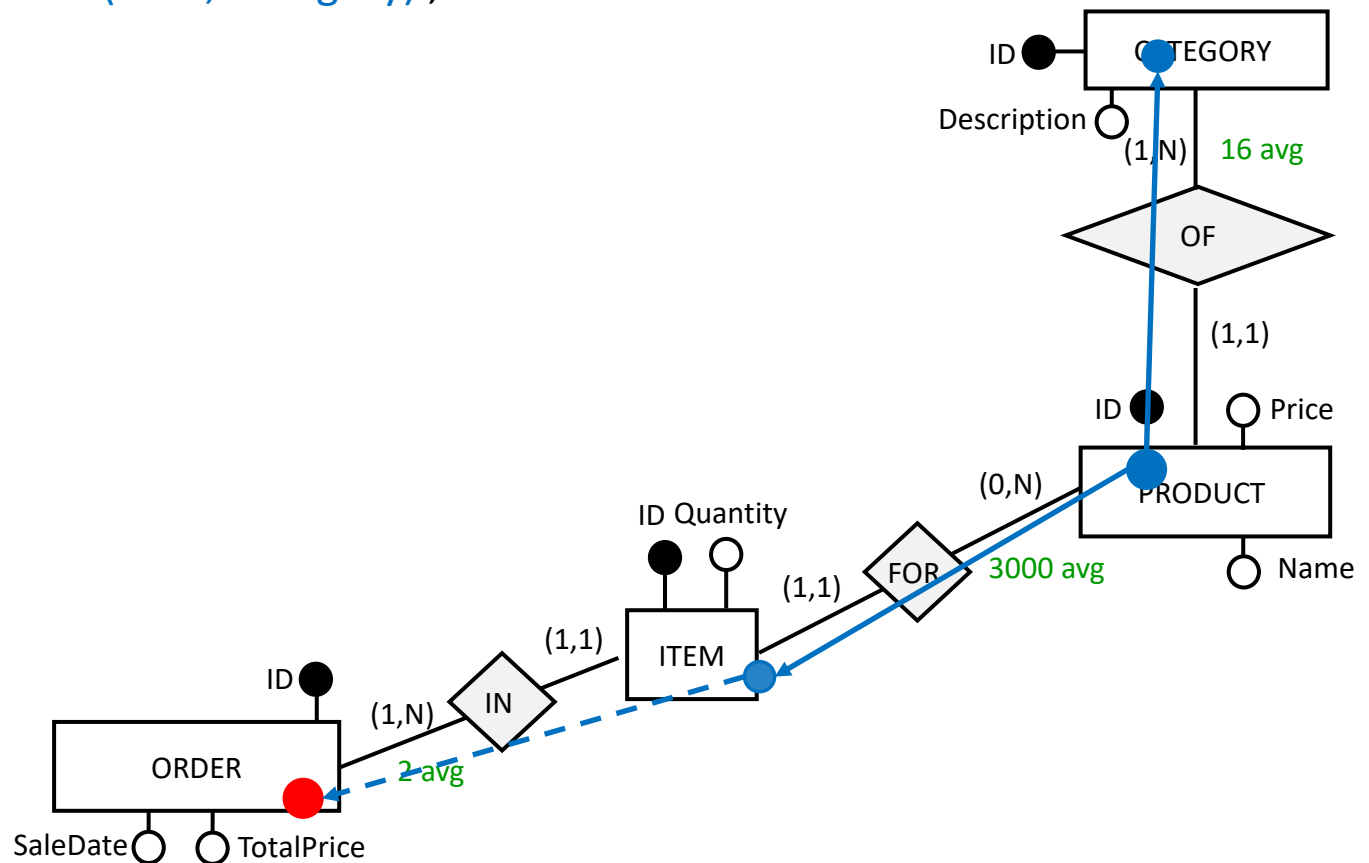
Solution 3

Collections: **Order** < **Item** < **Product** < **Category**



Solution 4

Collections: **Product** < (Item, Category) , **Order**



Exercise 2

1. Load the SmallOnlinestore DB from DUMP (form virtuale)
 - `mongorestore --db SmallOnlinestore <dumppath>`
2. Write the queries to create the collection Product for solution 4.
3. Estimate the size of the solutions
4. Estimate the cost of the workload (Q3 & Q4) in both the solutions 3 and 4

Step 2: DB creation

```
db.getSiblingDB("SmallOnlinestore").product.aggregate([
    {$lookup: { from: 'category', localField: 'category', foreignField: 'id', as: 'cat' } },
    {$project: { _id: 1, id: 1, price: 1, supplier: 1, description:1, category: { $arrayElemAt:
["$cat", 0] }, }},
    {$lookup: { from: 'item', localField: 'id', foreignField: 'product', as: 'items' } },
    { $out: { db: "SmallOnlinestore", coll: "s4.product" } },
]);
```

Step 3: Table of Volumes

SOLUTION 3	len	ND	NP
Order	1020	1200000	433085
TOTAL			433085

SOLUTION 4	len	ND	NP
Product	330235*	800	93477
Order	350	1200000	148608
TOTAL			242085

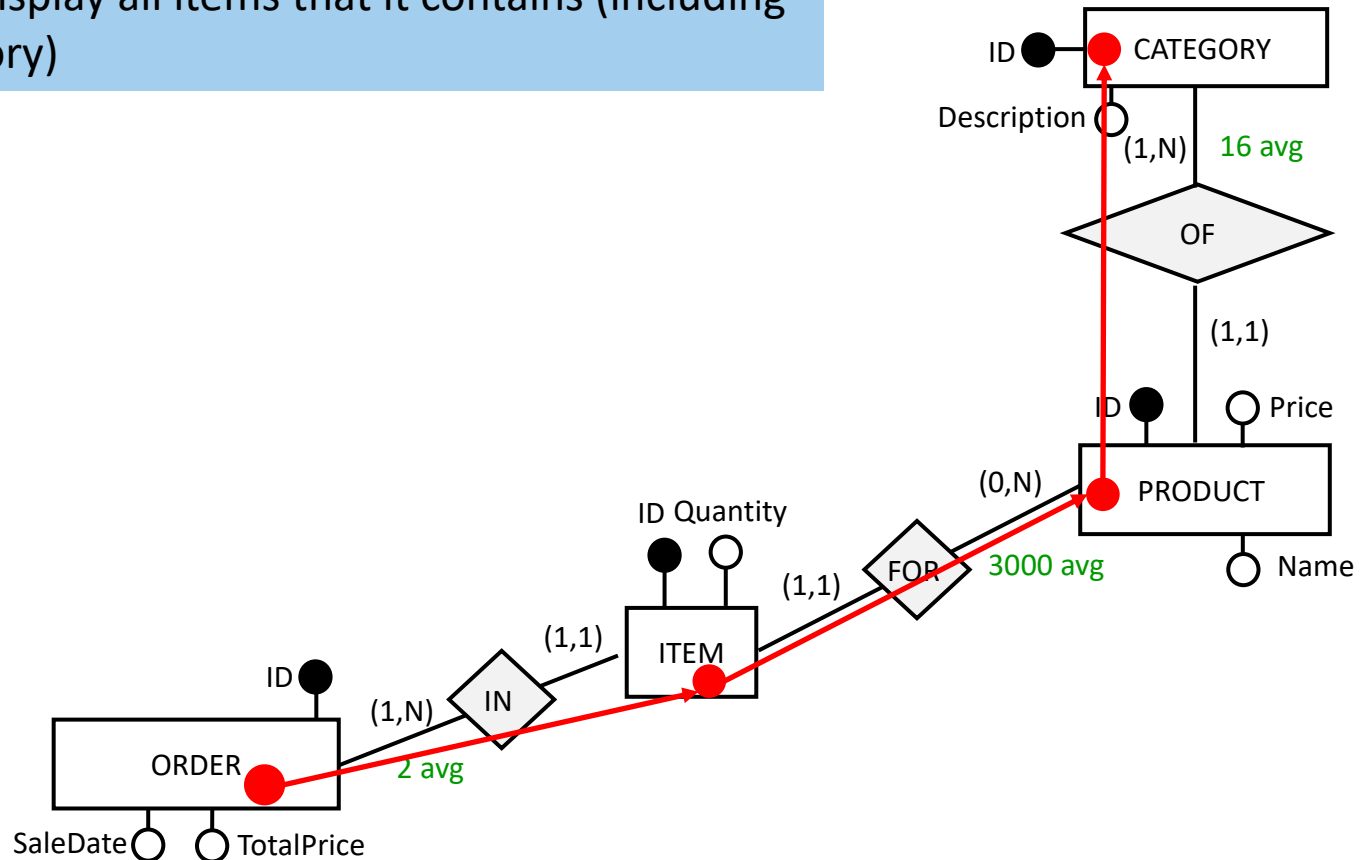
*Product takes **DP**=117 pages $\lceil 330235/(4096 \times 0,69) \rceil$

Step 4: Q3 cost Solution 3

Q3: Given a order, display all items that it contains (including price and category)

$ED = ED_{ORDER} = 1$

$EP = EP_{ORDER} = 1$

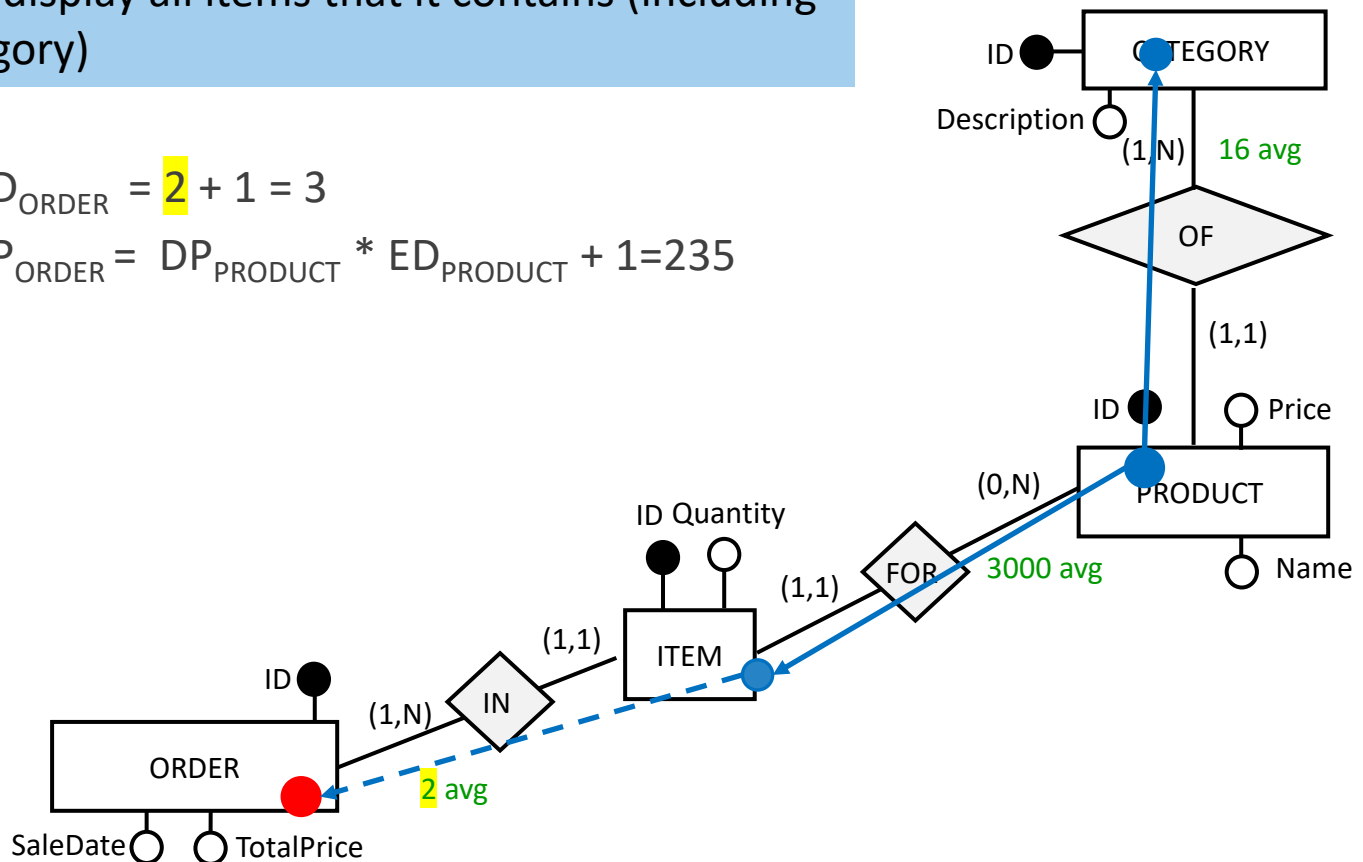


Step 4: Q3 cost Solution 4

Q3: Given a order, display all items that it contains (including price and category)

$$ED = ED_{\text{PRODUCT}} + ED_{\text{ORDER}} = 2 + 1 = 3$$

$$EP = EP_{\text{PRODUCT}} + EP_{\text{ORDER}} = DP_{\text{PRODUCT}} * ED_{\text{PRODUCT}} + 1 = 235$$



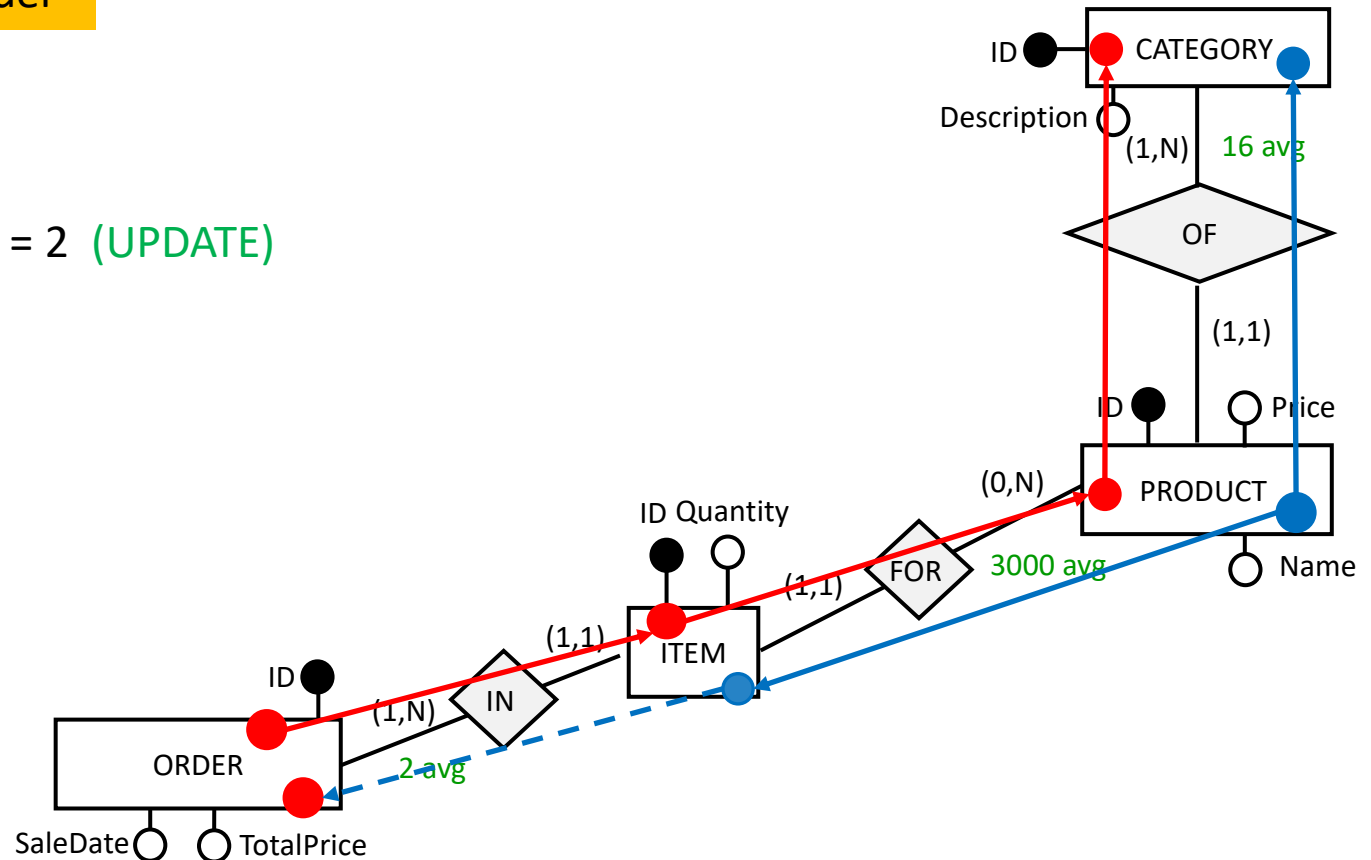
Step 4: Q4 cost Solution 3&4

Q4: Update a order

$$ED = ED_{\text{ORDER}} = 1$$

$$EP = EP_{\text{ORDER}} = 1$$

$$\text{TotalCost} = 2 * EP = 2 \text{ (UPDATE)}$$



Summary

	Max len	NP	Tot ED	Tot EP
Solution 1	1020	433085	3003	2986
Solution 2	330235	242085	3002	2985